

# ML Exam Cheatsheet — Weeks 1–7

---

## DATA LOADING

```
import pandas as pd, numpy as np
df = pd.read_csv('file.csv') # CSV
df = pd.read_csv('file.csv', na_values='?') # CSV with missing marker
df = pd.read_excel('file.xlsx', sheet_name='Sheet1') # Excel
df = pd.read_csv('file.txt', sep='\t') # TXT (tab-separated)
df.head() | df.info() | df.describe()
```

## MISSING VALUES

```
df.isnull().sum() # count missing per column
df.dropna() # remove rows with any NaN
df.dropna(subset=['col']) # remove rows where col is NaN
df['col'].fillna(df['col'].mean()) # fill with mean
df['col'].fillna(df['col'].median()) # fill with median
df['col'].fillna('Unknown') # fill with constant
df.fillna(method='ffill') # forward fill
```

## FILTER & SELECT

```
df[df['col'] > 5] # filter by value
df[(df['a'] > 1) & (df['b'] == 'Yes')] # multiple conditions
df[['col1', 'col2']] # select columns by name
df.drop(columns=['col']) # drop column
df.drop(columns='Unnamed: 0')
df.loc[df['col']=='Yes', 'col2'] # loc: label-based
df.iloc[0:5, 1:3] # iloc: position-based
df['new'] = (df['col'] == 'Yes').astype(int) # binary encode
```

## AGGREGATE

```
df.groupby('col').mean()
df.groupby('col').agg({'a': 'mean', 'b': 'sum'})
df['col'].value_counts()
df.corr() | sns.heatmap(df.corr(), annot=True)
```

---

## WEEK 1 — LINEAR REGRESSION

```

import statsmodels.formula.api as smf
from sklearn.linear_model import LinearRegression

# Simple Linear Regression (statsmodels)
result = smf.ols('sales ~ TV', data=df).fit()
result.summary()           # full table: coef, p-value, R2
result.params              # intercept + coefficients
result.rsquared            # R2
result.rsquared_adj        # Adjusted R2
result.fvalue              # F-statistic
result.conf_int()          # 95% confidence intervals
result.predict(pd.DataFrame({'TV': [100]}))

# Multiple Linear Regression
result = smf.ols('sales ~ TV + radio + newspaper', data=df).fit()

# Simple Linear Regression (sklearn)
X = df[['TV']].values      # must be 2D
y = df['sales'].values
model = LinearRegression().fit(X, y)
model.coef_ | model.intercept_
model.predict([[100]])

# Residual plot
plt.scatter(result.fittedvalues, result.resid)
plt.axhline(y=0, color='r', linestyle='--')

# Confidence intervals (statsmodels summary_table)
from statsmodels.stats.outliers_influence import summary_table
st, data, ss2 = summary_table(result, alpha=0.05)
predict_mean_ci_low, predict_mean_ci_upp = data[:,4:6].T

```

---

## WEEK 2 — POLYNOMIAL, INTERACTIONS, CATEGORICAL

```

from sklearn.preprocessing import PolynomialFeatures

# Polynomial (sklearn)
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)
model = LinearRegression().fit(X_poly, y)

# Polynomial (statsmodels formula)
result = smf.ols('mpg ~ horsepower + np.power(horsepower, 2)', data=df).fit()

# Interaction terms
result = smf.ols('sales ~ TV * radio', data=df).fit() # TV + radio + TV:radio

# Categorical variable with dummy coding
result = smf.ols('wage ~ age + C(education)', data=df).fit()
pd.get_dummies(df['col'], drop_first=True) # manual dummies

# VIF (collinearity)

```

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
vif = [variance_inflation_factor(X, i) for i in range(X.shape[1])]
```

## WEEK 3 — CLASSIFICATION

```
from sklearn.linear_model import LogisticRegression
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,
QuadraticDiscriminantAnalysis
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report, roc_curve, auc

# Logistic Regression (statsmodels)
result = smf.logit('default_yes ~ balance', data=df).fit()
result.summary()
result.predict(df)          # predicted probabilities

# Logistic Regression (sklearn)
lr = LogisticRegression()
lr.fit(X, y)
lr.predict(X)               # class labels
lr.predict_proba(X)[: ,1]  # probabilities for class 1

# LDA
lda = LinearDiscriminantAnalysis()
lda.fit(X, y)
lda.predict(X)
lda.predict_proba(X)[: ,1]
lda.priors_

# LDA with custom threshold
probs = lda.predict_proba(X)[: ,1]
y_pred = (probs >= 0.3).astype(int) # default threshold is 0.5

# LDA with custom priors
lda2 = LinearDiscriminantAnalysis(priors=[0.9, 0.1]).fit(X, y)

# QDA
qda = QuadraticDiscriminantAnalysis().fit(X_train, y_train)

# KNN Classifier
knn = KNeighborsClassifier(n_neighbors=3).fit(X_train, y_train)
knn.predict(X_test)

# Evaluation
cm = confusion_matrix(y_test, y_pred) # [[TN,FP],[FN,TP]]
print(classification_report(y_test, y_pred)) # precision, recall, f1

sensitivity = cm[1,1] / (cm[1,1] + cm[1,0]) # TP/(TP+FN)
specificity = cm[0,0] / (cm[0,0] + cm[0,1]) # TN/(TN+FP)

# ROC / AUC
fpr, tpr, _ = roc_curve(y_test, probs)
roc_auc = auc(fpr, tpr)
plt.plot(fpr, tpr, label=f'AUC={roc_auc:.2f}')
```

```
plt.plot([0,1],[0,1], 'k--')

# Train/Test split by condition
train_bool = df['Year'] < 2005
X_train, X_test = X[train_bool], X[~train_bool]
y_train, y_test = y[train_bool], y[~train_bool]
```

## WEEK 4 — RESAMPLING METHODS

```
from sklearn.model_selection import train_test_split, KFold, cross_val_score,
cross_val_predict
from sklearn.metrics import mean_squared_error
from sklearn.utils import resample

# Validation Set (random split)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5,
random_state=1)
model.fit(X_train, y_train)
mse = mean_squared_error(y_test, model.predict(X_test))

# Sample by index
train_df = df.sample(196, random_state=1)
test_df = df[~df.isin(train_df)].dropna(how='all')

# K-Fold CV
kf = KFold(n_splits=10, shuffle=False)
scores = cross_val_score(model, X, y, scoring='neg_mean_squared_error', cv=kf)
mse = np.mean(np.abs(scores))

# LOOCV
loocv = KFold(n_splits=len(X), shuffle=False)
scores = cross_val_score(model, X, y, scoring='neg_mean_squared_error', cv=loocv)

# Polynomial CV loop
for i in range(1, 6):
    poly = PolynomialFeatures(degree=i)
    X_p = poly.fit_transform(X)
    scores = cross_val_score(LinearRegression(), X_p, y,
                             scoring='neg_mean_squared_error', cv=kf)
    print(f"Degree {i}: MSE={np.mean(np.abs(scores)):.2f}")

# Classification CV
predicted = cross_val_predict(LogisticRegression(), X, y, cv=5)
accuracy_score(y, predicted)

# Bootstrap
for i in range(100):
    sample = df.sample(len(df), replace=True)
    result = smf.logit('y ~ x', data=sample).fit(dis=0)
    bootstrap_se = df_params.std()
```

## WEEK 5 — DIMENSION REDUCTION & REGULARISATION

```

from sklearn.preprocessing import scale
from sklearn.decomposition import PCA
from sklearn.linear_model import Ridge, RidgeCV, Lasso
from sklearn.cross_decomposition import PLSRegression

# PCA
pca = PCA()
X_reduced = pca.fit_transform(scale(X))
np.cumsum(np.round(pca.explained_variance_ratio_, 4) * 100) # cumulative variance %

# PCR (choose M components via CV, then fit)
regr = LinearRegression()
for i in range(1, 17):
    scores = cross_val_score(regr, X_reduced[:, :i], y,
                             scoring='neg_mean_squared_error', cv=kf)
    mse.append(-scores.mean())

# PCR on train/test
pca2 = PCA()
X_tr = pca2.fit_transform(scale(X_train)) # fit+transform on train
X_te = pca2.transform(scale(X_test))[:, :5] # transform only on test
regr.fit(X_tr[:, :5], y_train)
pred = regr.predict(X_te)

# PLS
pls = PLSRegression(n_components=9)
pls.fit(scale(X_train), y_train)
pls.predict(scale(X_test))
pls.x_weights_

# Ridge CV
rcv = RidgeCV(alphas=np.linspace(0.01, 100, 1000), cv=10)
rcv.fit(X, y)
rcv.alpha_ # best alpha

# Lasso (find best alpha manually)
alphas = np.linspace(0.0001, 0.1, 1000)
errors = [mean_squared_error(y_test,
                             Lasso(a, max_iter=100000).fit(X_train, y_train).predict(X_test))
          for a in alphas]
best_alpha = alphas[np.argmin(errors)]
ls = Lasso(alpha=best_alpha, max_iter=100000).fit(X, y)
ls.coef_ # 0 = variable excluded (LASSO feature selection)

```

---

## WEEKS 6–7 — NON-LINEAR METHODS

```

import statsmodels.api as sm
from patsy import dmatrix
import scipy.interpolate as si
from pygam import LinearGAM, LogisticGAM, s, f

```

```

# Polynomial Regression (statsmodels)
X4 = PolynomialFeatures(4).fit_transform(df.age.values.reshape(-1,1))
fit = sm.GLS(df.wage, X4).fit()
fit.summary()
age_grid = np.arange(df.age.min(), df.age.max()).reshape(-1,1)
X_test = PolynomialFeatures(4).fit_transform(age_grid)
pred = fit.predict(X_test)

# ANOVA to choose polynomial degree
from statsmodels.stats.api import anova_lm
mod1 = smf.ols('wage ~ age', data=df).fit()
mod2 = smf.ols('wage ~ age + np.power(age,2)', data=df).fit()
mod3 = smf.ols('wage ~ age + np.power(age,2) + np.power(age,3)', data=df).fit()
anova_lm(mod1, mod2, mod3)      # p<0.05 means higher degree needed

# Polynomial logistic (wage > 250)
y = (df.wage > 250).astype(int)
clf = sm.GLM(y, X4, family=sm.families.Binomial(sm.families.links.Logit())).fit()

# Step Functions (piecewise constant)
df_cut, bins = pd.cut(df.age, 4, retbins=True, right=True)
dummies = sm.add_constant(pd.get_dummies(df_cut))
dummies = dummies.drop(dummies.columns[1], axis=1) # drop first category
fit = sm.GLM(df.wage, dummies.astype(int)).fit()
# shortcut via formula:
smf.ols('wage ~ pd.cut(age, 4)', data=df).fit()

# Regression Splines (patsy)
t_x = dmatrix("bs(df.age, knots=(25,40,60), degree=3, include_intercept=False)",
              {"df.age": df.age}, return_type='dataframe')
fit1 = sm.GLM(df.wage, t_x).fit()

t_x2 = dmatrix("bs(df.age, df=6, include_intercept=False)",
               {"df.age": df.age}, return_type='dataframe') # auto knots

t_x3 = dmatrix("cr(df.age, df=4)", # natural spline
               {"df.age": df.age}, return_type='dataframe')

pred = fit1.predict(dmatrix("bs(age_grid, knots=(25,40,60), include_intercept=False)",
                           {"age_grid": age_grid}, return_type='dataframe'))

# Smoothing Spline (scipy)
order = np.argsort(x)
spl = si.LSQUnivariateSpline(x[order], y[order], t=[25,40,60])
plt.plot(x[order], spl(x[order]))

# GAM - Classification
gam = LogisticGAM().fit(X, y)
gam.accuracy(X, y)
gam.summary()

# GAM - Regression
gam = LinearGAM(s(0) + s(1) + f(2)).gridsearch(X, y) # s=spline, f=factor

# Partial dependence plots (GAM)
for i, ax in enumerate(axes):

```

```

XX = gam.generate_X_grid(term=i)
pdep, confi = gam.partial_dependence(term=i, width=.95)
ax.plot(XX[:,i], pdep)

# KNN Regression
from sklearn.neighbors import KNeighborsRegressor
knn = KNeighborsRegressor(n_neighbors=9).fit(X_train, y_train)
knn.predict(X_test)

# Find best K via GridSearchCV
from sklearn.model_selection import GridSearchCV
model = GridSearchCV(KNeighborsRegressor(),
                     {"n_neighbors": list(range(1,100))}, cv=5)
model.fit(X_train, y_train)
model.best_params_ # {'n_neighbors': N}

# Pipeline for CV over polynomial degrees
from sklearn.pipeline import Pipeline
for degree in range(1, 11):
    pipe = Pipeline([("poly", PolynomialFeatures(degree, include_bias=False)),
                     ("lr", LinearRegression())])
    scores = cross_val_score(pipe, X, y, cv=10, scoring='neg_mean_squared_error')
    print(degree, -np.mean(scores))

```

## QUICK REFERENCE

| Model        | Import                                                                   | Key call                                                 |
|--------------|--------------------------------------------------------------------------|----------------------------------------------------------|
| Linear Reg   | <code>sklearn.linear_model.LinearRegression</code>                       | <code>.fit(X,y)</code> <code>.predict(X)</code>          |
| Logistic Reg | <code>sklearn.linear_model.LogisticRegression</code>                     | <code>.fit(X,y)</code><br><code>.predict_proba(X)</code> |
| LDA          | <code>sklearn.discriminant_analysis.LinearDiscriminantAnalysis</code>    | <code>.priors_</code>                                    |
| QDA          | <code>sklearn.discriminant_analysis.QuadraticDiscriminantAnalysis</code> |                                                          |
| KNN Clf      | <code>sklearn.neighbors.KNeighborsClassifier(n_neighbors=k)</code>       |                                                          |
| KNN Reg      | <code>sklearn.neighbors.KNeighborsRegressor(n_neighbors=k)</code>        |                                                          |
| Ridge        | <code>sklearn.linear_model.RidgeCV(alphas=..., cv=10)</code>             | <code>.alpha_</code>                                     |
| Lasso        | <code>sklearn.linear_model.Lasso(alpha=a, max_iter=100000)</code>        | <code>.coef_</code>                                      |
| PCA          | <code>sklearn.decomposition.PCA()</code>                                 | <code>.fit_transform(scale(X))</code>                    |
| PLS          | <code>sklearn.cross_decomposition.PLSRegression(n_components=k)</code>   |                                                          |
| Poly         | <code>sklearn.preprocessing.PolynomialFeatures(degree=d)</code>          | <code>.fit_transform(X)</code>                           |
| Metric       | Code                                                                     |                                                          |
| MSE          | <code>mean_squared_error(y_test, y_pred)</code>                          |                                                          |

| Metric                | Code                                                       |
|-----------------------|------------------------------------------------------------|
| R <sup>2</sup>        | <code>result.rsquared (statsmodels)</code>                 |
| Accuracy              | <code>accuracy_score(y, pred)</code>                       |
| Confusion matrix      | <code>confusion_matrix(y, pred) → [[TN,FP],[FN,TP]]</code> |
| Classification report | <code>classification_report(y, pred)</code>                |
| AUC                   | <code>auc(*roc_curve(y, probs)[:2])</code>                 |